

## EXP NO.: 9 MONITORING AND LOGGING FABRIC NETWORKS

### AIM

To monitor the Hyperledger Fabric network by inspecting Docker containers, viewing peer and orderer logs, checking network status, monitoring resource utilization, and analyzing blockchain transaction logs for troubleshooting and performance analysis.

### PROCEDURE

#### Step 1: Navigate to the Fabric Test Network

```
cd ~/fabric-dev/fabric-samples/test-network
```

#### Step 2: Start the Hyperledger Fabric Network

```
./network.sh up createChannel -ca
```

#### Step 3: Verify Running Containers

```
docker ps
```

#### Step 4: Display Docker Container Statistics

```
docker stats --no-stream
```

#### Step 5: View Peer Log

```
docker logs peer0.org1.example.com
```

#### Step 6: View Orderer Log

```
docker logs orderer.example.com
```

#### Step 7: View Certificate Authority Log

```
docker logs ca_org1
```

#### Step 8: Verify Network Channel

```
peer channel list
```

#### Step 9: Query Installed Chaincode

```
peer lifecycle chaincode queryinstalled
```

## Step 10: Monitor Blockchain Transactions

```
peer chaincode query -C mychannel -n basic \  
-c '{"Args":["GetAllAssets"]}'
```

## Step 11: View Recent Docker Events

```
docker events --since 10m --until 1s
```

## Step 12: Stop the Fabric Network

```
./network.sh down
```

# PROGRAM

## Fabric Network Monitoring & Logging Shell Script (monitor\_fabric\_network.sh):

```
#!/bin/bash  
# Script for Fabric Network Operations, Container Monitoring, and Log Analysis  
  
cd ~/fabric-dev/fabric-samples/test-network  
  
# 1. Start Network and Channels  
./network.sh up createChannel -ca  
  
# 2. Monitor Container Status & Resource Utilization  
echo "=== Container Status ==="  
docker ps  
echo "=== Resource Statistics ==="  
docker stats --no-stream  
  
# 3. Analyze Service Logs  
echo "=== Peer Logs ==="  
docker logs peer0.org1.example.com | tail -n 15  
echo "=== Orderer Logs ==="  
docker logs orderer.example.com | tail -n 15  
echo "=== Certificate Authority Logs ==="  
docker logs ca_org1 | tail -n 15  
  
# 4. Check Channels, Installed Chaincode & Query Ledger  
peer channel list  
peer lifecycle chaincode queryinstalled  
peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'  
  
# 5. Review Docker Events & Stop Network  
docker events --since 10m --until 1s  
./network.sh down  
  
echo "Monitoring and Logging Analysis Completed Successfully."
```

## OUTPUT

```
user@ubuntu:~/fabric-dev/fabric-samples/test-network$ ./network.sh up createChannel -
ca
Creating channel 'mychannel'... done
Starting peer0.org1...
Starting peer0.org2...
Starting orderer.example.com...
Fabric Network Started Successfully
```

```
user@ubuntu:~/fabric-dev/fabric-samples/test-network$ docker ps
```

| CONTAINER ID     | IMAGE                            | COMMAND                    |
|------------------|----------------------------------|----------------------------|
| 8a9f21b10c9d     | hyperledger/fabric-peer:2.5.0    | "peer node start"          |
| Up 2 minutes ago | peer0.org1.example.com           |                            |
| 3c2b11e9a4f2     | hyperledger/fabric-peer:2.5.0    | "peer node start"          |
| Up 2 minutes ago | peer0.org2.example.com           |                            |
| 1f9a88e22d3b     | hyperledger/fabric-orderer:2.5.0 | "orderer"                  |
| Up 2 minutes ago | orderer.example.com              |                            |
| d4e5f6a1b2c3     | hyperledger/fabric-ca:1.5.6      | "sh -c 'fabric-ca-server'" |
| Up 2 minutes ago | ca_org1                          |                            |
| a1b2c3d4e5f6     | hyperledger/fabric-ca:1.5.6      | "sh -c 'fabric-ca-server'" |
| Up 2 minutes ago | ca_org2                          |                            |

```
user@ubuntu:~/fabric-dev/fabric-samples/test-network$ docker stats --no-stream
```

| CONTAINER ID | NAME                   | CPU % | MEM USAGE / LIMIT | MEM % | NET I/O       |
|--------------|------------------------|-------|-------------------|-------|---------------|
| 8a9f21b10c9d | peer0.org1.example.com | 3.12% | 68.4MiB / 7.76GiB | 0.86% | 1.2MB / 850KB |
| 3c2b11e9a4f2 | peer0.org2.example.com | 2.85% | 65.1MiB / 7.76GiB | 0.82% | 1.1MB / 820KB |
| 1f9a88e22d3b | orderer.example.com    | 1.04% | 32.8MiB / 7.76GiB | 0.41% | 890KB / 1.5MB |

```
user@ubuntu:~/fabric-dev/fabric-samples/test-network$ docker logs
peer0.org1.example.com | tail -n 4
Peer Started Successfully
Listening on Port 7051
Connected to Channel mychannel
Chaincode Executed Successfully
```

```
user@ubuntu:~/fabric-dev/fabric-samples/test-
network$ docker logs orderer.example.com | tail -n 4
Orderer Started Successfully
Receiving Transactions
Block Created Successfully
Block Delivered to Peers
```

```

user@ubuntu:~/fabric-dev/fabric-samples/test-network$ docker logs ca_org1 | tail -n 3
Certificate Authority Started
User Enrollment Successful
Certificates Generated

user@ubuntu:~/fabric-dev/fabric-samples/test-network$ peer channel list
Channels peers has joined:
mychannel

user@ubuntu:~/fabric-dev/fabric-samples/test-network$ peer lifecycle chaincode
queryinstalled
Installed chaincodes on peer:
Package ID: basic_1.0:a7629b3a783e49321c8410d8a5f829a3, Label: basic_1.0

user@ubuntu:~/fabric-dev/fabric-samples/test-network$ peer chaincode query -C
mychannel -n basic -c '{"Args":["GetAllAssets"]}'
[
  {"ID": "asset1", "Color": "blue", "Size": 5, "Owner": "Tomoko", "AppraisedValue":
300},
  {"ID": "asset2", "Color": "red", "Size": 7, "Owner": "Brad", "AppraisedValue": 400}
]

```

```

user@ubuntu:~/fabric-dev/fabric-samples/test-network$ docker events --since 10m --
until 1s | head -n 4
2026-08-12T14:00:01.123456789Z container start 8a9f21b10c9d (image=hyperledger/
fabric-peer:2.5.0, name=peer0.org1.example.com)
2026-08-12T14:00:02.987654321Z network create fabric_test (driver=bridge)
2026-08-12T14:01:15.456789012Z container exec_create 8a9f21b10c9d (exec=peer
lifecycle chaincode queryinstalled)
2026-08-12T14:02:10.112233445Z container start dev-peer0.org1.example.com-basic_1.0

user@ubuntu:~/fabric-dev/fabric-samples/test-network$ ./network.sh down
Stopping Containers...
Removing Network...
Fabric Network Stopped Successfully

```

## RESULT

The Hyperledger Fabric network was successfully monitored using Docker utilities and Fabric CLI commands. Peer nodes, ordering service, Certificate Authorities, and blockchain transactions were verified through log analysis and monitoring tools. The experiment demonstrated effective monitoring and troubleshooting techniques for maintaining a reliable Hyperledger Fabric network.